What to eat! Requirements

Version 1.0 | October 27, 2026 | Juan Castro | Meta Global AI Developer Hackathon, Indie track

Overview

What to eat! is the middleman between you and your food decision. It is a chat: short questions, three real places on a map, and an app that gets better at reading you with every meal. Muse Spark (Meta Model API) runs the reasoning; Google Places supplies the restaurants; Supabase stores history and the learned profile.

Core rules

ID	Requirement	Phase
R1	Every message is short: one line. Recommendation cards show name, distance, price, and a one-line reason.	All
R2	Muse only ranks real places returned by Google Places. It never invents a restaurant.	1
R3	Log from day one: time, day of week, answers, companions, price level, distance, the top 3, and the final choice. Phase 2 analytics depend on this data.	1

Functional requirements

Onboarding (taste quiz)

ID	Requirement	Phase
FR1	About 5 taps: favorite cuisines, hard no's, spice level, typical budget.	1
FR2	Quiz asks whether the user mostly eats solo or with people. If with people, the user adds companions by name, which fills the companions table.	1

Chat and questions

ID	Requirement	Phase
FR3	The app is a chat. One question per message, drawn from a pool: craving or mood, budget, distance, dine-in vs takeout, who you're with.	1
FR4	Exactly 3 questions (3 messages) before the first round of recommendations. In Phase 2, Muse reviews history for the current context and picks the 3 questions it is least sure about; answers it can predict are pre-filled and editable.	1-2
FR5	Phase 1 fallback: simple rules choose the 3 questions.	1

Recommendations

ID	Requirement	Phase
FR6	Google Places nearby search from the current location, using the search radius defined in FR-R2.	1
FR7	Muse ranks candidates using answers, taste profile, and companion patterns, and returns the top 3, each with a one-line reason.	1
FR8	The top 3 appear as map pins and compact cards; tapping one opens directions.	1
FR9	The user confirms which place they chose, or none.	1
FR9b	After the first round, the conversation stays open as long as needed; the user can refine (cheaper, closer, something else) and get new picks.	1

Feedback

ID	Requirement	Phase
FR10	A local notification fires 2 hours after the last app activity, only once a decision has been made. New activity reschedules it. If ignored, the app asks on next open.	1
FR11	Overall rating first, then optional price, taste, service, and experience ratings and a short comment, one question per message.	1
FR12	A preference setting for feedback mode: text (default) or voice. Voice uses Muse Voice Transcribe and produces the same structured feedback.	3
FR13	After each feedback, Muse updates the user's taste profile summary.	2

Companions

ID	Requirement	Phase
FR14	The companions table tracks, per person: meal count, likely days of the week, and most likely cuisines together.	2
FR15	Selected companions influence which questions are asked and how places are ranked.	2

Insights

ID	Requirement	Phase
FR16	Charts: average spend by time of day, most common craving by time slot, distance traveled, meals by companion, top cuisines.	2
FR17	A one-line Muse-written takeaway at the top of the Insights screen.	2

Adaptive search radius

The search area should grow more accurate to the user as usage increases. Values below are proposed starting points, to be tuned against real usage data.

ID	Requirement	Phase
FR-R1	Record the straight-line distance from the session location to the chosen place for every confirmed meal.	1

ID	Requirement	Phase
FR-R2	Radius for a new search = the recency-weighted 80th percentile of past travel distances in a similar context (time slot, weekday or weekend, companions), clamped between 1 and 25 miles.	2
FR-R3	Cold start: use the quiz answer until 5 confirmed meals exist for that context.	1-2
FR-R4	If fewer than 8 candidates come back, expand the radius by 1.5x, up to two times.	2

Non-functional requirements

ID	Requirement	Phase
NFR1	Runs on iPhone through Expo Go; no App Store build required.	1
NFR2	Backend deployed to the cloud by the end of Phase 1, so the app works away from home Wi-Fi.	1
NFR3	API keys live only on the backend; every API request requires a valid Supabase JWT.	1
NFR4	First round of recommendations in under about 8 seconds after the third answer.	1

Architecture

Mobile: Expo (React Native, TypeScript). **Backend:** FastAPI (Python, uv). **Places:** Google Places API. **AI:** Muse Spark via Meta Model API; Muse Voice Transcribe for voice feedback. **Data and auth:** Supabase (Postgres).

Tables: users, taste_profiles, companions, sessions, session_companions, messages, feedback.

Phases and deliverables

Phase	Scope	Due
1	**Core loop** (Tue Oct 27 to Thu Oct 29). Deliverable: the app in daily use on iPhone by Thursday night. Quiz, chat with 3 questions, top 3 on a map, refinement, confirm, text feedback with notification, full logging, deployed backend.	Oct 29
2	**Learning and insights** (Fri Oct 30 to Sun Nov 1). Deliverable: the app visibly adapts to real data. Muse-chosen questions, adaptive radius, companion metrics, taste profile updates, Insights charts.	Nov 1
3	**Voice and demo** (Mon Nov 2 to Wed Nov 4). Deliverable: project submitted by 9:00 PM ET on Nov 4. Voice feedback option, edge cases, polish, README, demo video.	Nov 4

Risks

• Phase 1 is the tightest phase and includes a cloud deploy. Every day it slips removes a self-use day, which is the core of the demo. First cut if needed: FR9b moves to Phase 2.

• Voice is deliberately Phase 3. If time runs out, text feedback still delivers the full product.

• The radius thresholds (80th percentile, 1 to 25 miles, 8 candidates) are untested guesses and should be tuned after 2 to 3 days of data.